__/__/____

Unit. IV

Dynamic Programming Technique

Dynamic Programming - Rod cutting - Longest Common Subsequence - Travelling Sales person Problem - String Editing - Longest common Substring - Longest non decreasing Subsequence - Staircase Problem.

Dynamic Programming

→ is similar to divide and conquer is breaking down the problems into smaller and yet smaller possible and subproblems.

→ But divide and conquer these subproblems are not solved independently.

→ dynamic programming algorithms are used for solving optimization problems.

Overlapping Subproblems

→ Dynamic programming also combines solutions to subproblem.

→ It is mainly used where the solution of one subproblem is needed repeatedly.

→ The computed solutions are stored in a table, so that these don't have to be re computed.

→ This technique is needed where overlapping subproblem exists.

__/__/____

Examples of dynamic programming

→ Matrix chain Multiplication
→ Floyd Warshall Algorithm
→ 0-1 Knapsack problem
→ Longest common subsequence Algorithm
→ Travelling Salesman problem.

Divide and Conquer	Dynamic programming
1) breaks a problem into independent, non overlapping subproblems, solves them separately and combines their solutions	Solves problems with overlapping subproblems by storing and reusing solutions to subproblems to avoid recalculating them making it efficient for optimization problems.
2) Solves subproblems recursively and then combines their results to form the solution to the original problem	Stores the solutions to subproblems so they can be reused, preventing redundant calculations.
3) Can be less efficient if the same subproblems are solved multiple times	More efficient than standard divide and conquer for problems with overlapping subproblems due to the reuse of results.
4) (Eg): Merge Sort, Quick Sort, Binary Search	(Eg) Fibonacci Sequence, Knapsack problem, shortest path algorithms

__/__/____

Rod Cutting

→ The goal is to maximize the profit obtained by cutting a rod of a given length into smaller pieces and selling them.

→ You are provided with a rod of length n and an array of prices where price[i] represents the selling price of a rod of length i+1.

→ Let's define C(i) as the price of the optimal cut of a rod up until length 'i'.

→ Let V_K be the price of a cut at length 'K' to develop a solution.

(i) we define the smallest problem first and store their solution.

(ii) we increase the rod length & try all the cuts for that size of rod, taking the most profitable one.

(iii) we store the optimal solution for this sized price & build solutions to larger pieces from then in some sort of data structure.

$$C(i) = \max_{1 \le K \le i} \{ V_K + C(i-K) \}$$

(Eg)

Length	1	2	3	4
Price	1	5	8	9

Find max profit obtainable on cutting the rod into pieces & selling.

dp[i] = max (Price[j] + dp[i-j-1]) ← best value of the remaining rod.

- price[j] = price of the first piece (length j+1)

__/__/____

(Eg) n = 8

price [] = { 1, 5, 8, 9, 10, 17, 17, 20 }

That means:

Length	1	2	3	4	5	6	7	8
price	1	5	8	9	10	17	17	20

Posible cuts

For length 1:

dp[1] = price[1] + dp[0] dp[1] = 1

= 1 + 0 = 1

For length 2:

dp[2]

= price[1] + dp[1] = 1 + 1 = 2 dp[2] = 5

= price[2] + dp[0] = 5 + 0 = 5

For length 3:

possible cuts

1 + remaining (2) = 1 + 5 = 6

2 + 1 " (1) = 5 + 1 = 6 dp[3] = 8

3 + " (0) = 8 + 0 = 8

For length 4:

1 + 3 => 1 + 8 = 9

2 + 2 => 5 + 5 = 10

3 + 1 => 8 + 1 = 9 dp[4] = 10

4 + 0 => 9 + 0 = 9

For length 5:

1 + 4 => 1 + 10 = 11

2 + 3 => 5 + 8 = 13 dp[5] = 13

3 + 2 => 8 + 5 = 13

4 + 1 => 9 + 1 = 10

5 + 0 => 10 + 0 = 10

__/__/____

For length 6' Price

1 + 5 => 1 + 13 = 14

Length 2 + 4 => 5 + 10 = 15

3 + 3 => 8 + 8 = 16 dp[6] = 17

4 + 2 => 9 + 5 = 14

5 + 1 => 10 + 1 = 11

6 + 0 => 17 + 0 = 17

For length 7

1 + 6 => 1 + 17 = 18

2 + 5 => 5 + 13 = 18

3 + 4 => 8 + 10 = 18

4 + 3 => 9 + 8 = 17 dp[7] = 18

5 + 2 => 10 + 5 = 15

6 + 1 => 17 + 1 = 18

7 + 0 => 17 + 0 = 17

For length 8

1 + 7 => 1 + 18 = 19

2 + 6 => 5 + 17 = 22

3 + 5 => 8 + 13 = 21

4 + 4 => 9 + 10 = 19 dp[8] = 22

5 + 3 => 10 + 8 = 18

6 + 2 => 17 + 5 = 22

7 + 1 => 17 + 1 = 18

8 + 0 => 20 + 0 = 20

For length 8 best value 22

cut into 2 + 6 (price[1] + dp[6])

=> 5 + 17 = 22

Time complexity $O(n^2)$ → Because for each length i we loop through all smaller cuts.

__/__/____

Longest Common Subsequence:

Algorithm

LCS (X, Y, m, n)

Create a table L[0..m][0...n]

For i=0 to m

For j=0 to n

If i==0 or j==0

L[i][j]=0

Else if X[i-1]==Y[j-1]

L[i][j]=L[i-1][j-1]+1

Else

L[i][j]=max(L[i-1][j], L[i][j-1])

Return L[m][n]

→ Given two strings text1 & text2 return the length of their longest common subsequence. If there is no common subsequence return 0.

→ A subsequence of a string is a new string generated from the original string with some characters deleted without changing the relative order of the remaining characters.

(Eg)

text1 = 'ace' , text2 = "abcde"

		0	0	a	b	c	d	e
	0	0	0	0	0	0	0	0
a	0	0+1=1	✗	1	1	1	1	not match take maximum
c	0	1	✗	1+1=2	2	2	2	match take diagno value+1
e	0	1	✗	2	2	2	3	

__/__/____

Longest Common Subsequence

↳ Data comprossion in one data set to another data set

String editing → Data edit & replace with one dataset

1) X = BACDB Y = BDCB

X/Y		0	B	D	C	B
	0	0	0	0	0	0
B	0	0+1=1	1	1	1	0+1=1
A	0	1	1	1	1	1
C	0	1	1	1+1=2	2	2
D	0	1	1+1=2	2	2	2
B	0	0+1=1	2	2	2	2+1=3

BDB

not matched → maximum value (Cross)

match → diagnol value +1

Travelling Salesman Problem

__/__/____

→ Consider a graph G(V, E) where V is set of vertices and E is set of edges and assume that source vertex is 1.

→ Tour is defined as a simple path that starts and ends at source vertex 1.

Key points

→ Tour starts from any of the edge (1, k) where $k \in V - \{1\}$

→ There is a path from vertex k to 1 through each vertex in $V - \{1, k\}$

→ $G(i, S)$ be the cost of a shortest path starting at vertex i going through all vertices in S and terminating at vertex 1.

→ Minimum cost tour: The function

$$g(1, V - \{1\}) = \min_{2 \le k \le n} \{ c_{1k} + g(k, V - \{1, k\}) \}$$

when $|V - \{1\}| = n - 1$

→ Above relation can be generalized as

$$g(i, S) = \min_{j \in S} \{ c_{ij} + g(j, S - \{j\}) \}$$ where

$$S \subset n-1, \; i \neq 1, \; i \neq S, \; 1 \neq S$$

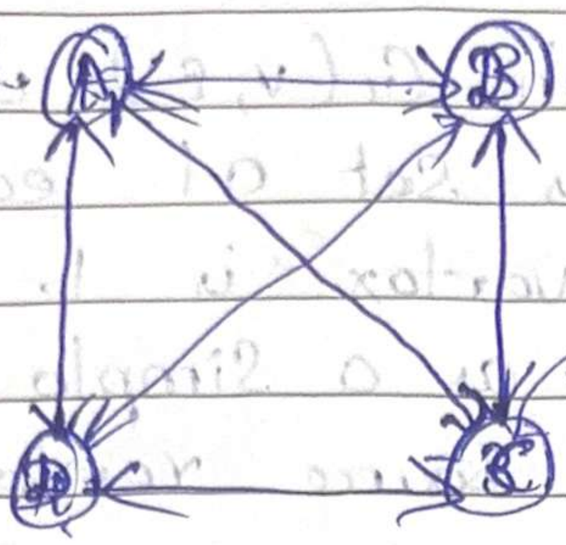
→ Base case $g(i, \phi) = c_{i1}$ for $1 \le i \le n$ and $i \neq 1$

i → Starting vertex S → Possible vertex

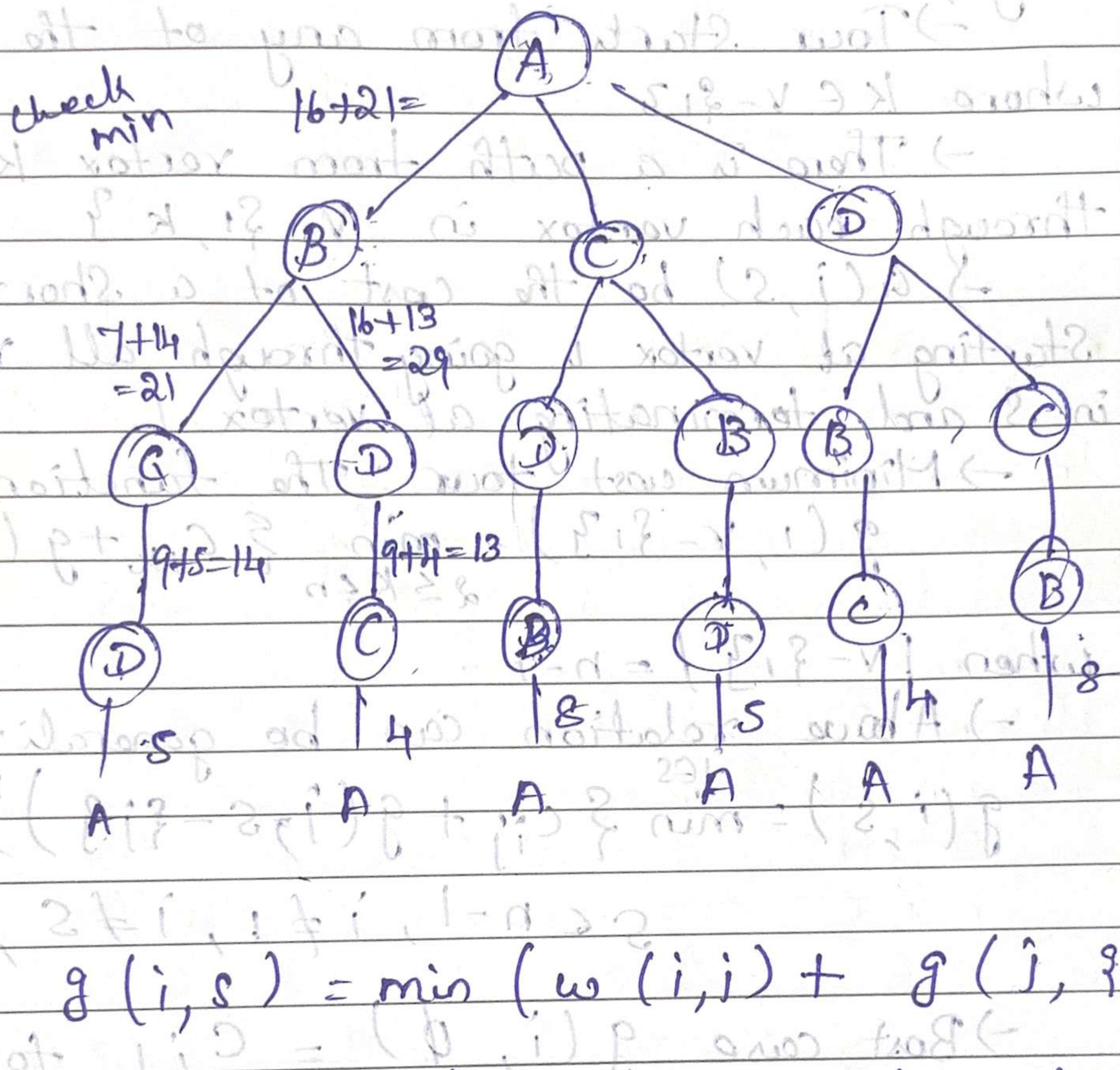
j → End vertex

__/__/____

Example.



	A	B	C	D
A	0	16	11	6
B	8	0	13	16
C	4	7	0	9
D	5	12	2	0



$$g(i,S) = \min\left(w(i,j) + g(j, \{S-j\})\right)$$

$$g(A,\{B,C,D\}) = \min\left(w(A,B) + g(B,\{C,D\})\right.$$
$$= 16+22 = 38$$
$$= \min\left(w(A,C) + g(C,\{B,D\})\right.$$
$$\min\left(w(A,D) + g(D,\{B,C\})\right.$$

$$g(B, \{C, D\}) = w(B, C) + g(C, \{D\})$$
$$= 13 + 14 = 27$$
$$w(B, D) + g(D, \{C\})$$
$$= 16 + 6 = 22$$

$$\Rightarrow g(C, \{D\}) = w(C, D) + g(D, \phi)$$
$$= 9 + 5 = 14$$

$$g(D, \{C\}) = w(D, C) + g(C, \phi)$$
$$= 2 + 4 = 6$$

$$g(C, \{B, D\}) = \min(w(C, B) + g(B, \{D\}))$$
$$= 7 + 21 = 28$$
$$= \min(w(C, D) + g(D, \{B\}))$$
$$9 + 20 = 29$$

$$g(B, \{D\}) = \min(w(B, D) + g(D, \phi))$$
$$10 + 5 = 15$$

$$g(D, \{B\}) = \min(w(D, B) + g(B, \phi))$$
$$12 + 8 = 20$$

$$g(D, \{B, C\}) = \min(w(D, B) + g(B, \{C\}))$$
$$= 12 + 7 = 19$$
$$w(D, C) + g(C, \{B\})$$
$$= 2 + 15 = 17$$

$$g(B, \{C\}) = \min(w(B, C) + g(C, \phi)) = 13 + 4 = 17$$

$$g(C, \{B\}) = \min(w(C, B) + g(B, \phi)) = 7 + 8 = 15$$

$$A \rightarrow D \rightarrow C \rightarrow B = 2\}$$

String Editting

There are two strings

$X = x_1, x_2, x_3 \ldots x_n$

$Y = y_1, y_2, y_3 \ldots y_n$

where x_i & y_j $(i = 1, 2, 3 \ldots n$ & $j = 1, 2, 3 \ldots m)$ are members of alphaset.

→ Problem is to transform X into Y using a sequence of edit operations on X.

Edit operations are.

→ insert

→ delete

→ change

→ Cost value is associated with each operator.

→ Cost can be represented as follows.

$D(x_i)$ ⇒ cost of deleting the Symbol x_i

$I(y_j)$ ⇒ cost of inserting the symbol y_j

$C(x_i, y_j)$ ⇒ cost of changing x_i of X into y_j

Dynamic Programming Solution

Stated as follows

→ define cost (i, j) → It is the minimum cost of any edit sequence for transforming $x_1, x_2, x_3 \ldots x_i$ into $y_1, y_2, y_3 \ldots y_j$

→ Compute $c(i, j)$ for each i & j

→ Then, cost (n, m) is the cost of an optimal edit sequence.

Rule for transformation

__/__/____

Rule 1:

If $i = j = 0$ then

$cost(i, j) = 0$

Since two sequences are empty & identical.

Rule 2:

if $j = 0$ & $i > 0$ then

X can be transformed into Y by a Sequence of delete.

[Eg:- X = aa Y = ϕ to transform x to Y we need to do two delete]

Cost can be represented as

$cost(i, 0) = cost(i-1, 0) + D(x_i)$

Rule 3:

If $i = 0$ & $j > 0$ then x can be transformed into Y by a Sequence of instruction

[Eg: X = ϕ, Y = aa to transform x to Y, we need to do two inserts]

Cost can be represented as

$cost(0, j) = Cost(0, j-1) + I(y_j)$

Rule : 4

if $i \neq 0$ & $j \neq 0$ then $x_1, x_2 \ldots x_i$ can be transformed $y_1, y_2 \ldots y_j$ in any one of three ways.

$$cost(i, j) = \begin{cases} 0 & i - j = 0 \\ cost(i-1, 0) + D(x_i) & j = 0, i > 0 \\ cost(0, j-1) + x(y_j) & i = 0, j > 0 \\ cost(i, j) & i > 0, j > 0 \end{cases}$$

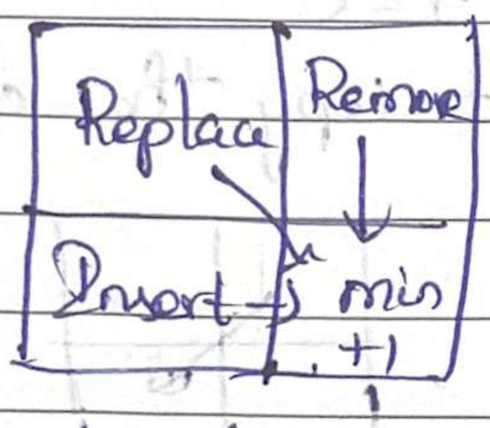
__/__/____

	Null	a	b	c	f	g
Null	0 →	1 →	2 →	3 →	4 →	5.
a	↓ 1 →	0 →	1 →	2 →	3 →	4
d	↓ 2	↓ 1	1 →	2 →	3 →	4
c	↓ 3	↓ 2	2	1 →	2 →	3
e	↓ 4	↓ 3	3	2	2 =	3
g	↓ 5	↓ 5	4	3	3	2

adcceg
abcfg

→ 2 operations

If $r \neq c$



If $r = c$

Just copy the diagonal element

```
for (i=0; i<len(Str1)+1 ; i++)
 {
  for (j=0 ; j<len(Str2)+1 ; j++)
  {   if(i==0 && j==0)
        T[i][j]=0;
  else if (i==0)
        T[i][j]==T [i][j-1]+1;
  else if (j==0)
    T[i][j]=T[i-1][j]+1;
```

__/__/____

2) Str1 = Kitten Str2 = Sitting

	Null	S	i	t	t	i	n	g
Null	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	3
n	6	6	5	4	3	3	2	3

OP

Str1 = k i t t e n
↓ Replace ↓ Replace ↓ Insert
Str2 = S i t t i n g

Replace	Remove
Insert →	min +1

Longest Common Substring

→ is problem that finds the longest Substring of two Strings

→ There is one difference between the longest common Subsequence and the longest common Substring.

→ In the case of Substring, all the elements in the Substring must be in contiguous in a original String and the order of the elements in the Substring Sould be Same as in the String.

→ In the case of Subsequence we can miss out some elements which means that it is not mandatory that the elements in the Substring Should be contiguous.

Algorithm

Consider two Strings S1 and S2.

if (S1[i] == S2[j])
T[i][j] = T[i-1][j-1] + 1
else
T[i][j] = 0

(Eg)

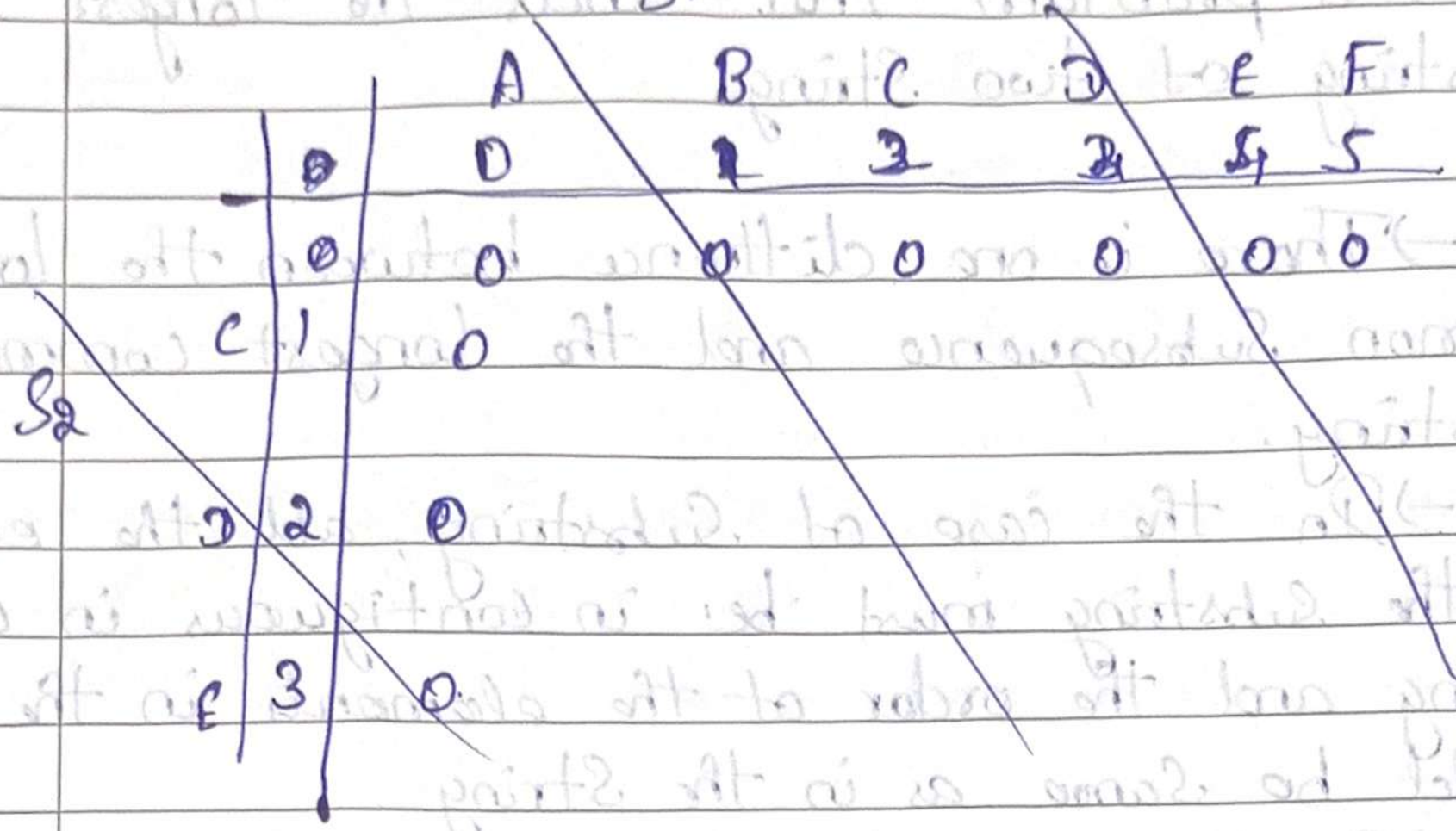
S1 = a b c d a f

S2 = z b c d f

$$C(i,j) = \begin{cases} 0 & i=0, j=0 \\ 1 + C[i-1][j-1] & \text{if } S_1[i] == S_2[j] \\ 0 & S_1[i] != S_2[j] \end{cases}$$

__/__/____

S_1 = 'ABCDEF' ; S_2 = 'CDE'



	A	B	C	D	E	F
	0	1	2	3	4	5
	0	0	0	0	0	0
C 1	0	0	1+0=1	0	0	0
D 2	0	0	0	1+1=2	0	0
E 3	0	0	0	0	2+1=3	0

max length = 3

Substring = CDE

not match

take min

match

diagonal +1

__/__/____

Stair Case Problem

→ You are climbing a staircase.
→ It takes n steps to reach the top
→ Each time you can either climb 1 step or 2 steps.
Find the total number of distinct ways to reach the top.

Define a recursive function climbStairs(n) that returns the number of ways to reach the nth step

Base case : climbStairs(0) = 1 & climbStairs(1)
Recursive step : climbStairs(n) = claimbStairs(n-1) + claimbStairs(n-2)

Algorithm

```
int climbStairs (int n)
{
  int result [n+1];
  result [0] = 1;
  result [1] = 1;
  for (int i=2; i<=n; i++)
  {
    result [i] = result [i-1] + result [i-2];
  }
  return result [n];
}
```

Problem Statement:
Given an integer N representing the number Stairs, find the total number of distinct ways to reach the Nth Stair Starting from the 0th Stair. At each Step one can climb

either 1 or 2 stairs.

Dynamic Programming Approach

This problem exhibits optimal substructure and overlapping subproblems making it suitable for dynamic programming.

Define Subproblem → Let dp[i] be the number of distinct ways to reach the ith Stair.

Base cases:

→ To reach the 0th Stair (the starting point) there is 1 way (do nothing).

So dp[0] = 1

→ To reach the 1st Stair there is 1 way (take one 1-Step), So dp[1] = 1

Recurrence Relation:

To reach the ith Stair one could have come from the $(i-1)^{th}$ Stair by taking a 1 Step or from the $(i-2)^{th}$ Stair by taking a 2 Step.

Therefore the total number of ways to reach the ith Stair is the sum of ways to reach the $(i-1)^{th}$ Stair and ways to reach the $(i-2)^{th}$ Stair.

$$dp[i] = dp[i-1] + dp[i-2] \text{ for } i \geq 2$$